# StockPilot
# Inventory Management System

*Complete project plan and Week 1 workbook*

Portfolio preparation for a DPS Telecom Entry-Level Software Developer application | October 2026

Tech stack: PHP 8+, MySQL 8+, vanilla JavaScript, HTML, CSS | Recommended timeline: 6 weeks | 10–14 hours/week

PURPOSE • Build a credible internal-business web application while learning the underlying technologies yourself. This is a learning roadmap, not a completed application.

## Contents


1. Goal and business scenario
2. Requirements and business rules
3. Interface wireframes
4. Database schema
5. Suggested folder structure
6. Six-week roadmap
7. Week 1 daily plan
8. Coding exercises and sample snippets
9. Mentorship workflow and exit checklist


## 1. Goal and business scenario

StockPilot serves a small telecommunications equipment business that holds network switches, replacement power supplies, cables, and spare parts. Employees need to locate items, track quantities, record incoming deliveries and outgoing issues, see low-stock items, and identify who made changes.

Professional outcome: an operational inventory web application, clean source repository, setup documentation, sample data, testing notes, and a short demonstration suitable for a developer interview.

Role alignment: practice PHP, relational MySQL data, HTML/CSS/JavaScript, debugging, documentation, and internal business workflows. Embedded C and hardware-focused skills are separate preparation topics and are not covered by this project.

## 2. Requirements and business rules

### Functional requirements

| Feature | Scope | Stage |
|---|---|---|
| **Inventory dashboard** | Totals, low-stock counts, recent activity | MVP |
| **Products** | Create, view, edit, archive; unique SKUs | MVP |
| **Categories** | Group equipment by type | MVP |
| **Stock in/out** | Record quantity movements | MVP |
| **Search/filter** | SKU, name, category | MVP |
| **Low-stock alerts** | Quantity at or below reorder level | MVP |
| **Authentication** | Secure sign-in/out | Phase 2 |
| **Authorization** | Admin and staff roles | Phase 2 |

| | | |
|---|---|---|
| **Movement history** | Auditable records with user and timestamp | Phase 2 |
| **CSV reports** | Export products or movement records | Phase 2 |
| **Suppliers** | Vendor contact data | Optional |
| **Barcode scanning** | Fast product lookup | Stretch |

### Non-functional requirements

- Security: PDO prepared statements, password hashing, output escaping, CSRF protection, server-side validation and authorization.
- Integrity: inventory balance and movement record must update in the same database transaction.
- Reliability: reject negative stock; validate input; show usable error messages.
- UX: responsive layout, labeled form inputs, clear success/errors, keyboard-friendly navigation.
- Maintainability: reusable PHP functions, readable folder structure, documented setup, Git history.

### Inventory business rules

1. SKU must be unique across products.
2. Quantity and reorder level must be nonnegative integers.
3. Every stock receipt or issue must produce a movement record.
4. Normal product editing cannot directly change quantity; use the inventory workflow.
5. Archive a product with transaction history rather than deleting its audit trail.
6. Low stock means quantity <= reorder_level.
7. Only authorized users can change inventory.

## 3. Interface wireframes

These are schematic offline wireframes, not implemented screens.

```
STOCKPILOT  |  Dashboard                                  Admin
+-----------------+-------------------------------------------------+
| Dashboard       | Inventory overview                              |
| Products        | [128 Products] [2450 Units] [8 Low] [3 Out]     |
| Stock movements |                                                 |
| Reports         | Low-stock alerts                                |
| Users           | Power Supply 12V ..... 2 units                  |
|                 | Ethernet Cable ..... 5 units                    |
+-----------------+-------------------------------------------------+
```

```
PRODUCTS                                  [+ Add Product]
[ Search by name or SKU ..... ] [Category: All]
+--------------------+----------+----------+--------------+
| Name               | SKU      | Quantity | Actions      |
+--------------------+----------+----------+--------------+
| Network Switch     | NET-001  | 25       | View / Edit  |
| Power Supply       | PWR-002  | 2 (LOW)  | View / Edit  |
| Ethernet Cable     | CAB-003  | 120      | View / Edit  |
+--------------------+----------+----------+--------------+
```

Additional views: Add/Edit Product form (name, SKU, category, price, reorder level); Stock In/Out form (product, movement type, quantity, note); movement history table (date, product, type, units, user).

## 4. Database schema

Core relationship map: categories (1) → products (many); products (1) → stock_movements (many); users (1) → stock_movements (many).

| Table | Columns / constraints |
|---|---|

| | |
|---|---|
| **categories** | id INT PK AUTO_INCREMENT; name VARCHAR(100) NOT NULL UNIQUE; description TEXT NULL |
| **products** | id INT PK; category_id INT FK; sku VARCHAR(50) UNIQUE NOT NULL; name VARCHAR(150) NOT NULL; description TEXT; price DECIMAL(10,2) >= 0; quantity INT >= 0 DEFAULT 0; reorder_level INT >= 0 DEFAULT 5; is_active BOOLEAN DEFAULT TRUE; created_at DATETIME; updated_at DATETIME |
| **users** | id INT PK; name VARCHAR(100); email VARCHAR(255) UNIQUE; password_hash VARCHAR(255); role (admin/staff); created_at DATETIME |
| **stock_movements** | id INT PK; product_id INT FK; user_id INT FK; movement_type (IN/OUT); quantity INT > 0; note TEXT NULL; created_at DATETIME |

Implementation notes: use InnoDB foreign keys, indexed SKU/category fields, and server-side checks. For stock-out, lock the product row (SELECT ... FOR UPDATE), ensure enough quantity, then update quantity and insert a movement in one transaction. Default initial stock handling must also create an auditable movement when the full workflow is added.

## 5. Suggested folder structure

```
stockpilot/
├── app/
│   ├── config/database.php
│   ├── functions/{products.php, inventory.php, auth.php}
│   └── views/partials/{header.php, sidebar.php, footer.php}
├── public/
│   ├── index.php
│   ├── products.php
│   ├── product-create.php
│   ├── product-edit.php
│   ├── stock.php
│   └── assets/{css/style.css, js/app.js}
├── database/{schema.sql, seed.sql}
├── tests/
├── .gitignore
└── README.md
```

public/ is the web root; app/ contains private PHP code; database/ keeps reproducible SQL scripts; tests/ stores verification work. Later refactor to route/controller classes only as needed.

## 6. Six-week roadmap

| Week | Focus | Acceptance milestone |
|---|---|---|
| **1** | Local environment, PHP basics, static Product UI, MySQL schema and SQL exercises | Local page + initial database + Git history |
| **2** | PDO connection; read, add and edit products; validation | Working product CRUD |
| **3** | Stock-in/out flows, history, transactions, integrity checks | Consistent inventory quantities |
| **4** | Dashboard calculations, filters, alerts, report export | Useful operations dashboard |
| **5** | Login, roles, permissions, CSRF, secure sessions | Protected application |
| **6** | Tests, accessibility, polish, setup guide, demo and deployment | Interview-ready repository and demo |

## 7. Week 1 daily plan (Oct 19–25, 2026)

### Monday, Oct 19 — Development environment (1.5–2 h)

☐ Verify PHP, MySQL, Git and VS Code

☐ Create stockpilot and public/

☐ Serve first PHP homepage

☐ Initialize Git

Deliverable: Browser renders PHP-generated HTML.

### Tuesday, Oct 20 — PHP basics (2 h)

☐ Variables, arrays, loops, functions

☐ Build associative arrays for products

☐ Embed PHP in HTML

☐ Render five sample rows with foreach

Deliverable: Five PHP-array products appear in an HTML table.

### Wednesday, Oct 21 — Semantic HTML (1.5–2 h)

☐ Build sidebar, page title, search and table

☐ Use semantic table headings and form labels

☐ Populate placeholder rows

Deliverable: Unstyled but complete Products page.

### Thursday, Oct 22 — CSS and JS (2 h)

☐ Style with Flexbox/Grid

☐ Add low-stock visual cues

☐ Implement client-side filter for sample rows

☐ Check mobile layout

Deliverable: Responsive static table with working search.

### Friday, Oct 23 — MySQL fundamentals (2 h)

☐ Create stockpilot_db

☐ Define categories and products

☐ Seed five to ten products

☐ Practice SELECT, WHERE, ORDER BY, JOIN

Deliverable: SQL returns sorted and filtered products.

### Saturday, Oct 24 — Integration preparation (1.5–2 h)

☐ Read about PDO and prepared statements

☐ Diagram browser → PHP → database

☐ Document setup in README

☐ Commit progress

Deliverable: Project ready for Week 2 PDO integration.

### Sunday, Oct 25 — Review and cleanup (1 h)

☐ Run all acceptance checks

☐ Fix SQL and layout errors

☐ Commit Week 1

☐ Write 3 lessons + 2 open questions

Deliverable: Week 1 complete.

## 8. Coding exercises and sample snippets

### Exercise 1 — First PHP page

Understand server-side rendering: PHP executes on the server and emits HTML. Run these terminal checks:

```
php -v
mysql --version
git --version
mkdir stockpilot && cd stockpilot
mkdir public
php -S localhost:8000 -t public
```

Create public/index.php:

```
<?php
$appName = "StockPilot";
?>
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title><?php echo $appName; ?></title></head>
<body><h1>Welcome to <?php echo $appName; ?></h1></body>
</html>
```

Challenge: show the app name, a short application description, and the current year calculated in PHP. Verify http://localhost:8000.

### Exercise 2 — PHP arrays and foreach

```
<?php
$product = [
   "sku" => "NET-001",
   "name" => "Network Switch",
   "quantity" => 25,
   "price" => 129.99
];
echo $product["name"];
?>
```

☐ Create a list of five product arrays containing SKU, name, quantity and price.

☐ Render an HTML <tr> for each product using foreach.

☐ Mark products low-stock when quantity is 5 or less.

☐ Add a sixth product; confirm it appears without hard-coding another row.

### Exercise 3 — MySQL schema and queries

```
CREATE DATABASE stockpilot_db;
USE stockpilot_db;
CREATE TABLE categories (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100) NOT NULL UNIQUE
);
```

Finish the products table based on the schema above, insert at least three records (e.g., Network Switch NET-001, Power Supply PWR-002, Ethernet Cable CAB-003), and write SELECT queries to retrieve all products, low-stock products, and alphabetically sorted product names. Try a JOIN with categories.

Stretch: explain why UNIQUE, FOREIGN KEY, and DECIMAL matter. Do not copy a full answer until you have attempted the queries yourself.

# 9. Mentorship workflow and exit checklist

8. Explain: learn one concept with an example.
9. Practice: solve a focused small exercise.
10. Build: implement the feature independently.
11. Review: examine your code, errors, naming and design choices.
12. Commit: record working progress with meaningful Git messages.

## Week 1 acceptance checks

☐ PHP homepage runs locally.

☐ Products page shows five products generated from PHP arrays.

☐ Layout works on desktop and mobile.

☐ Vanilla JS filters the sample product table.

☐ MySQL database includes categories and products.

☐ You can run basic SQL SELECT, WHERE, ORDER BY and JOIN queries.

☐ README exists and all progress is committed to GitHub.

## Interview-ready explanations

- Why are category and product data stored in separate tables?
- How does PHP differ from browser JavaScript?
- How will PDO prepared statements protect queries?
- Why must stock movement creation and quantity updates be atomic?
- How does an archived product preserve history?
- What test would catch simultaneous stock-out errors?

Next step: finish Day 1, then review the PHP page and terminal output before moving to PHP arrays. Week 2 introduces database-backed CRUD only after the Week 1 foundation is verified.